

Support Vector Machine

Juan G. Colonna
jgcolonna@uea.edu.br

Support Vector Machine

Máquinas de Vetores de Suporte (SVMs) são uma classe particularmente poderosa e flexível de algoritmos supervisionados para classificação e regressão.

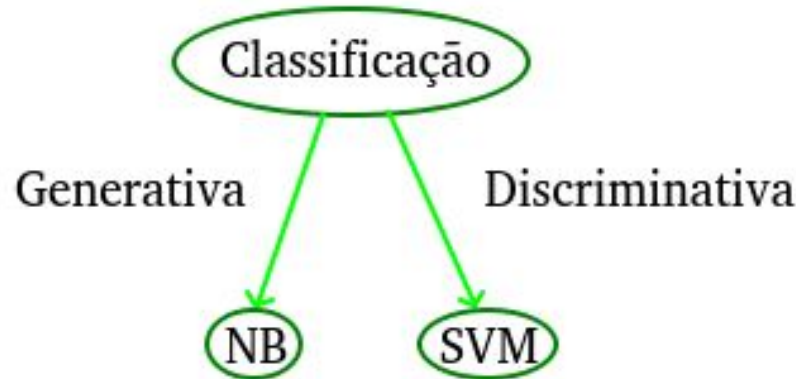
Existem diferentes versões:

1. Binary SVM
 - a. Multiclass SVM
2. Support Vector Regression
3. One-class SVM

Support Vector Machine

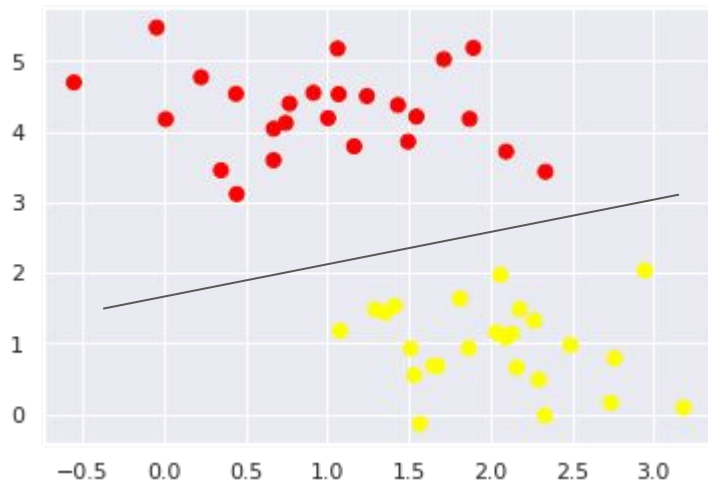
Na aula de classificação Bayesiana aprendemos um modelo simples (e variações) que descreve a distribuição subjacente de cada classe e usamos esses modelos para determinar probabilisticamente rótulos para novos pontos.

Ao invés disso, SVM é baseado no conceito de classificação discriminativa que, no lugar de modelar cada classe, simplesmente procura ajustar uma linha (ou curva) que divide as amostras de cada classes.



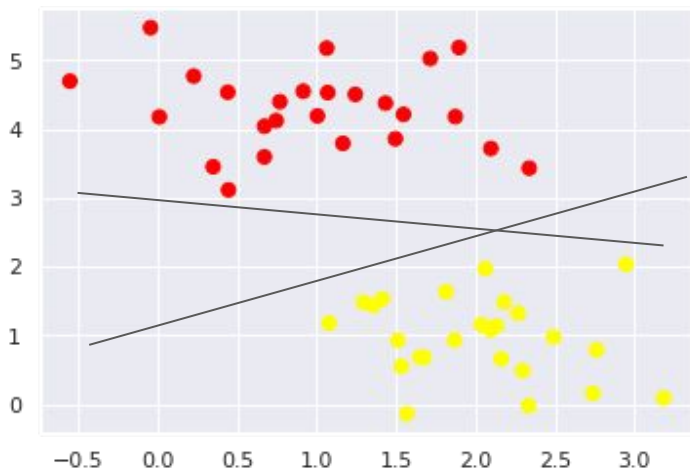
Support Vector Machine

Como exemplo disso, considere o caso simples de uma tarefa de classificação, em que as duas classes de pontos estão bem separadas:



Support Vector Machine

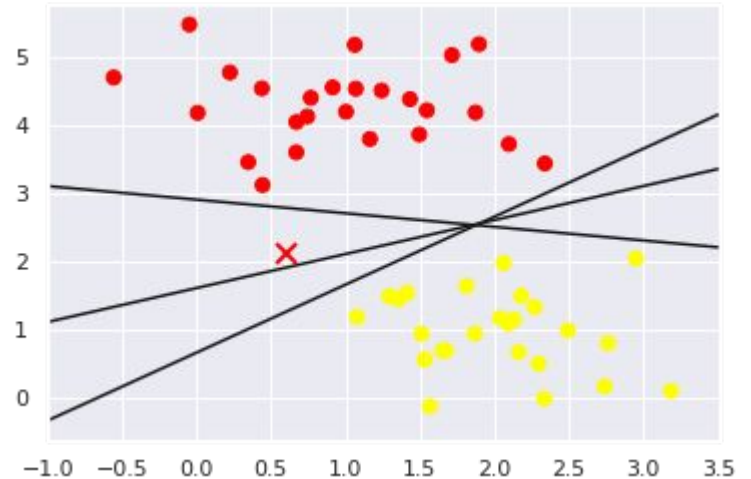
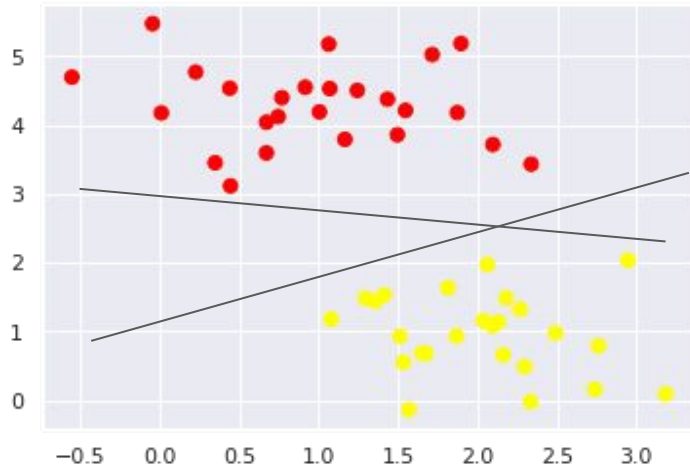
Um classificador discriminativo linear tentaria desenhar uma linha reta separando os dois conjuntos de dados e, assim, criar um modelo para classificação.



Mas, qual reta
seria melhor?

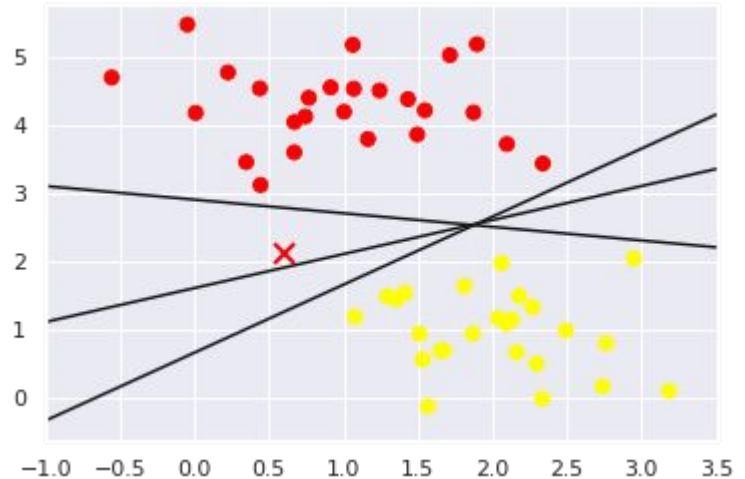
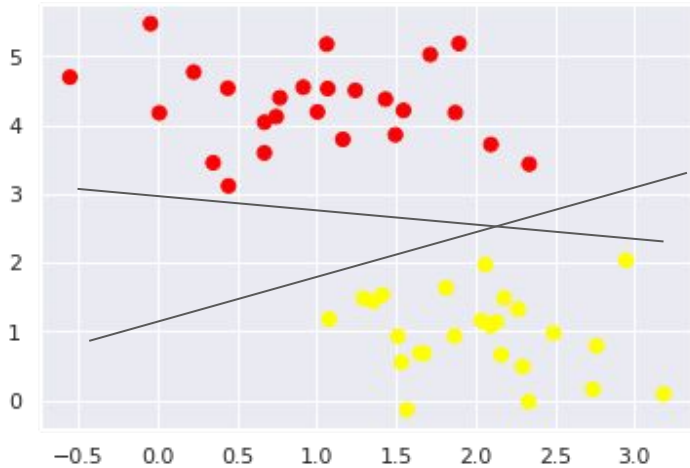
Support Vector Machine

Três retas diferentes distinguem perfeitamente essas amostras, mas dependendo de qual você escolher, um novo ponto de dados receberá um rótulo diferente!



Support Vector Machine

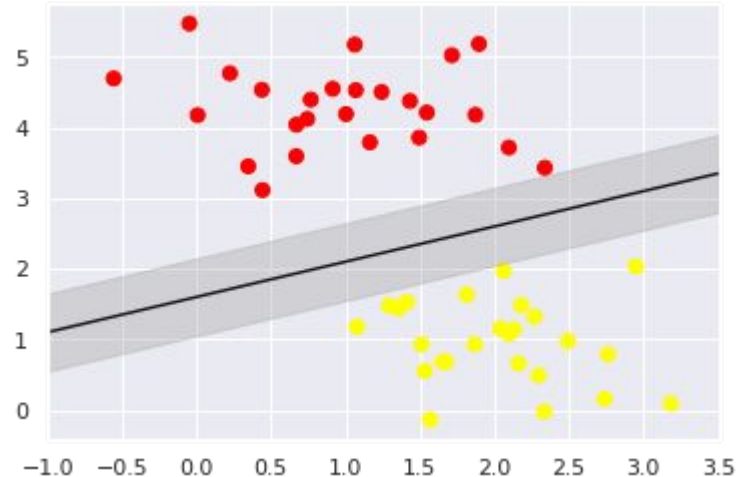
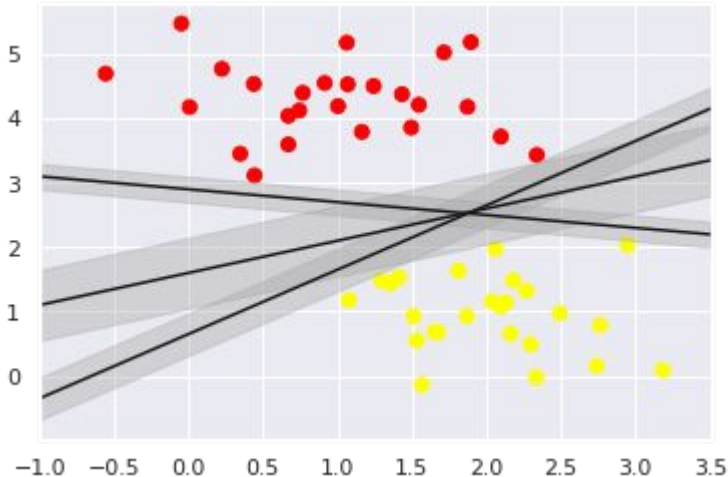
Evidentemente, nossa simples intuição de "traçar uma linha entre as classes" não é suficiente, e precisamos pensar um pouco mais a fundo.



Support Vector Machine

As SVM oferecem uma maneira de lidar com esse problema.

A intuição é a seguinte: em vez de simplesmente desenhar uma linha qualquer entre as classes, podemos escolher uma linha que maximiza a margem entre os pontos mais próximos das duas classes.



Support Vector Machine

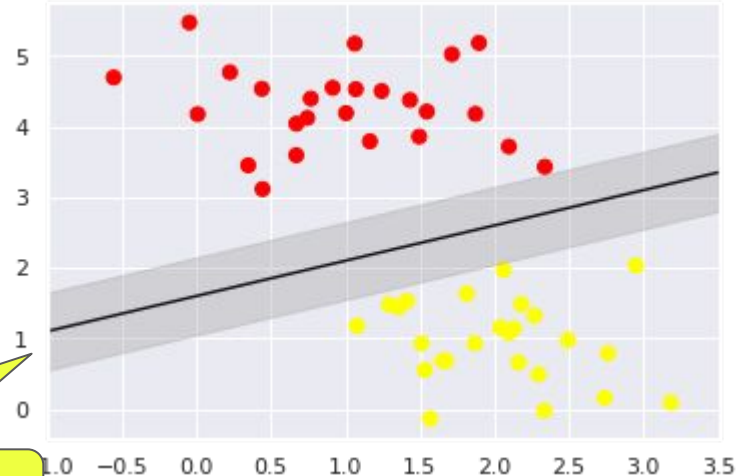
Em máquinas de vetores de suporte, a linha que maximiza essa margem é aquela que escolhemos como o modelo ótimo.

As máquinas de vetores de suporte são um exemplo desse estimador de margem máxima.

Exemplo usando Sklearn:

<https://colab.research.google.com/drive/1TIKzbvrFGkOWMSps3UhOJO8IfY6PCnV?usp=sharing>

g



$f(\mathbf{x})$ função de decisão, resultados positivos indicam uma classe, enquanto que resultados negativos indicam a outra classe.

Support Vector Machine

No Scikit-Learn SVM chama-se SVC (Support Vector Classifier) no caso de um problema de classificação, ou SVR (Regression) para um problema de regressão.

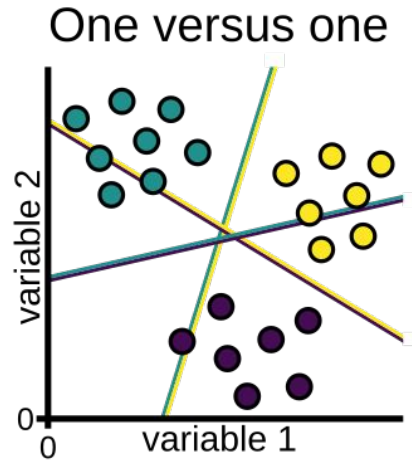
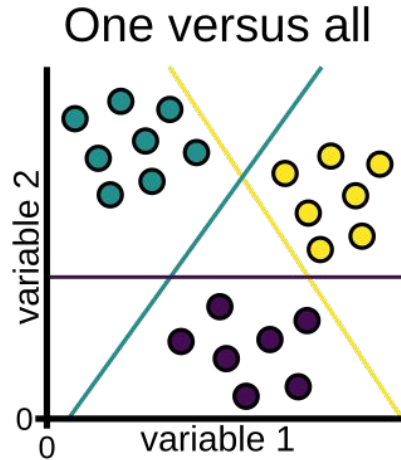
Os parâmetros mais relevantes são:

- C: parâmetro de regularização para evitar sobre ajuste (overfitting). Quanto maior C menor a regularização.
- kernel: tipo de kernel 'linear', 'poly', 'rbf', etc.
- degree: grau do polinômio se o kernel é 'poly'.
- gamma: coeficiente do kernel 'rbf'.
- decision_function_shape: estratégia One-versus-One ou One-versus-Rest.

Support Vector Machine

Naturalmente SVM possui a restrição de ser um classificador binário (unicamente duas classes).

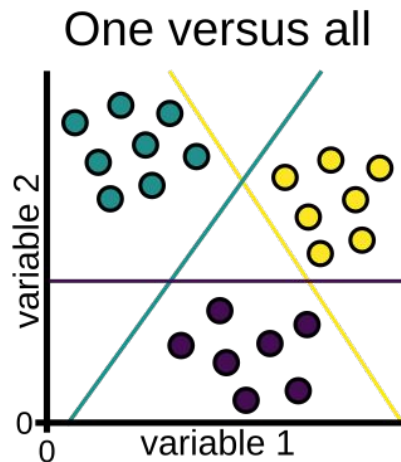
As estratégias One-versus-One ou One-versus-Rest servem para estender o classificador e criar um método multiclasse.



Support Vector Machine

One-vs-Rest

Classificador	Classe Positiva	Classes Negativas	Pontuação para X
svm1	C1	{C2, C3}	+0.9
svm2	C2	{C1, C3}	-1.2
svm3	C3	{C1, C2}	+0.1

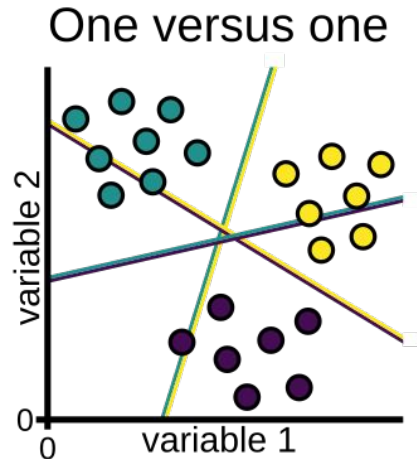


	c1	c2	c3
svm1	1	0	0
svm2	1	0	1
svm3	0	0	1
soma	2	0	2

votações podem ter empate, nesse caso podemos usar o maior score para tomar a decisão.

Support Vector Machine

One-vs-One



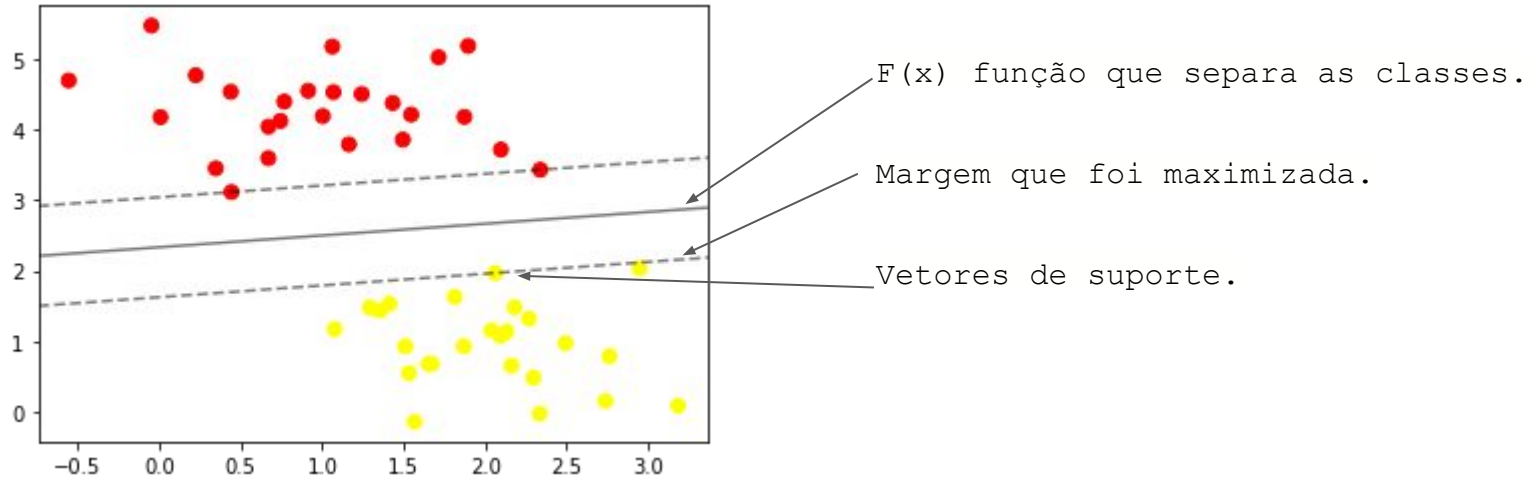
Classificador	Competição	Voto para X	Justificativa (Exemplo)
svm12	C1 vs. C2	C1	X parece mais com C1 do que C2
svm13	C1 vs. C3	C1	X parece mais com C1 do que C3
svm23	C2 vs. C3	C3	X parece mais com C3 do que C2

	c1	c2	c3
svm1	1	0	-
svm2	1	-	0
svm3	-	1	0
soma	2	1	0

votações

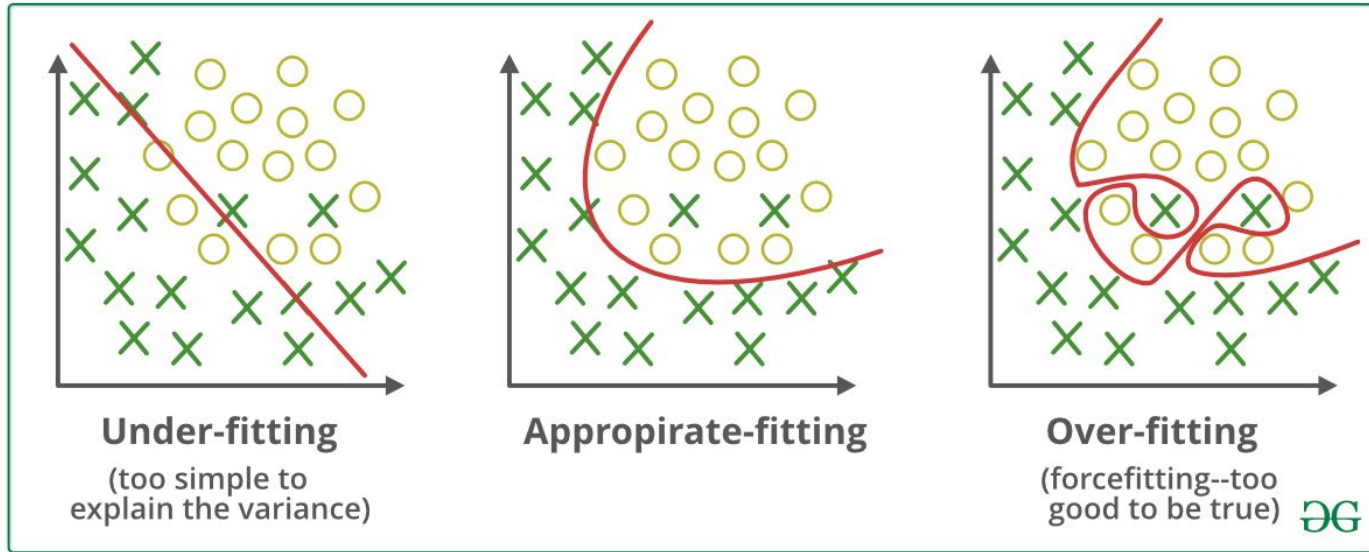
Support Vector Machine

No exemplo obtivemos:



Overfitting vs Underfitting

Controle pelo parâmetro C.

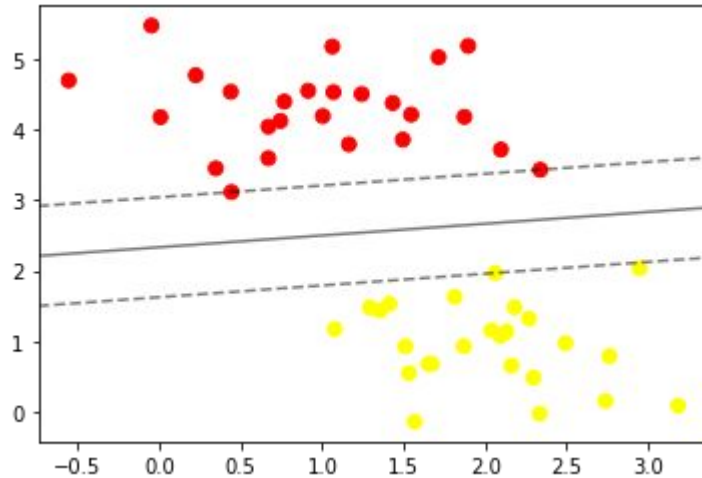


<https://www.geeksforgeeks.org/underfitting-and-overfitting-in-machine-learning/>

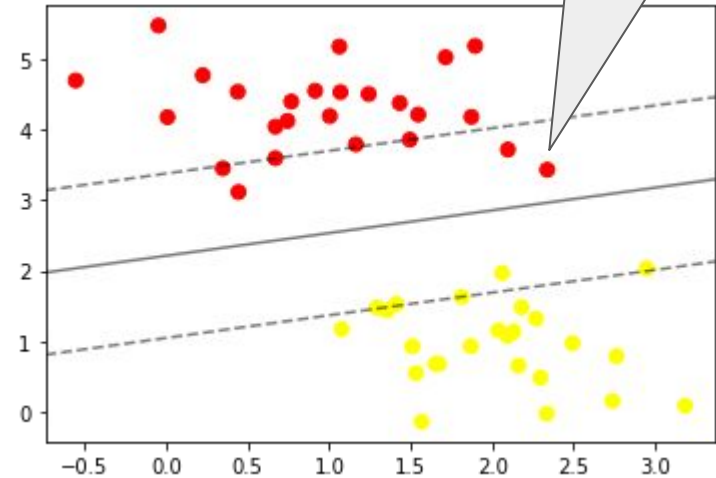
Support Vector Machine

No exemplo obtivemos:

$C = 1e10$



$C = 0.1$

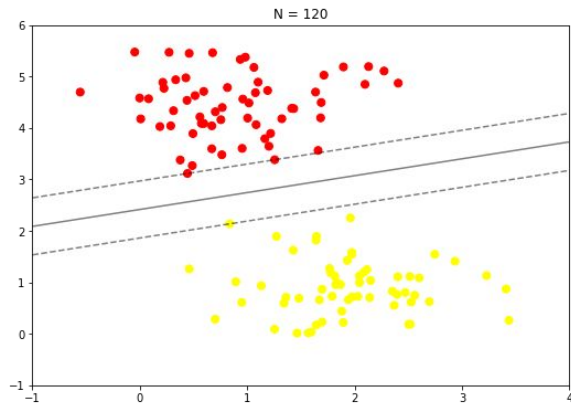
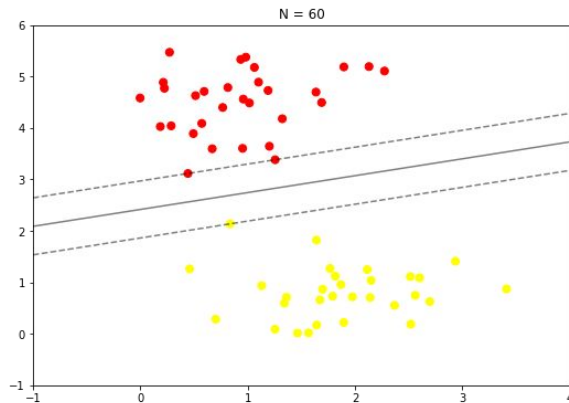


Se tolera que alguns pontos invadam a margem para não sobre ajustar o modelo.

Support Vector Machine

Um ponto chave para deste método é que uma vez descobertos os vetores de suporte apenas a posição dos vetores de suporte importa; quaisquer pontos mais afastados da margem não modificam o ajuste!

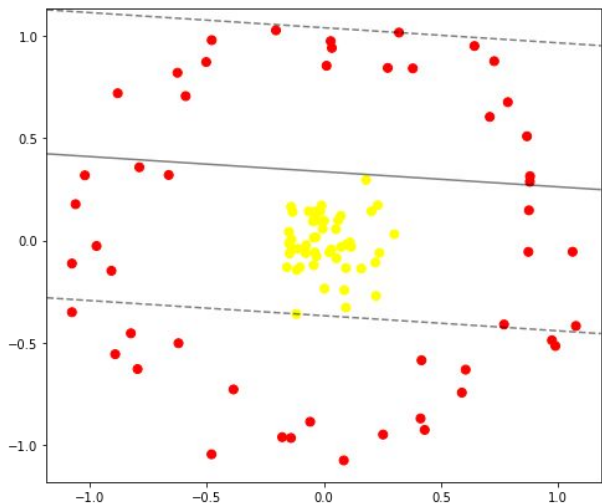
Tecnicamente, isso ocorre porque esses pontos não contribuem para a função de perda usada para ajustar o modelo, portanto, sua posição e valor não importam, desde que não invadam a margem.



SVM não linear

O SVM se torna extremamente poderoso quando é combinado com diferentes funções de kernels.

O objetivo do kernel é projetar nossos dados em um espaço vetorial de dimensão superior onde seja mais possível ajustar uma função linear para classificar dados não linearmente separáveis.

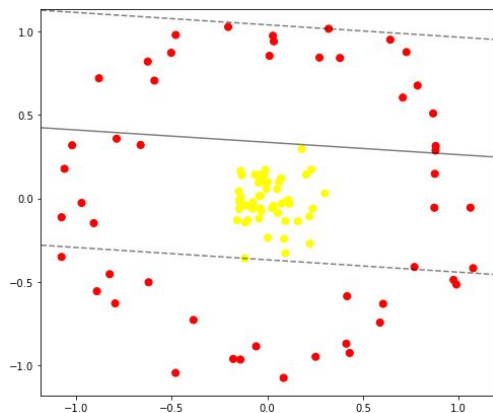


Neste dataset uma função linear nunca irá separar os dados de forma correta.

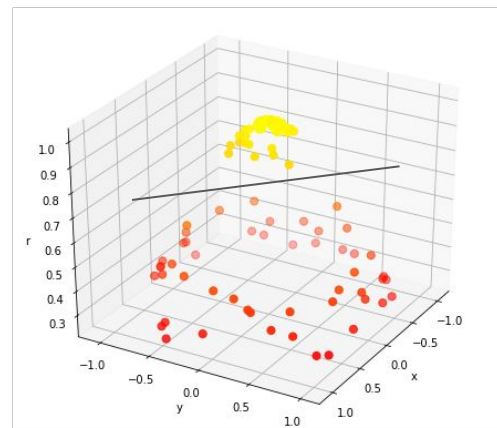
SVM não linear

Com a função de kernel podemos aumentar a dimensão dos dados (seria equivalente a adicionar uma nova coluna no dataset), de modo que um separador linear seria suficiente.

Por exemplo, uma projeção simples que poderíamos usar seria calcular uma função centrada no meio da classe amarela `r = np.exp(-(X ** 2).sum(1))`



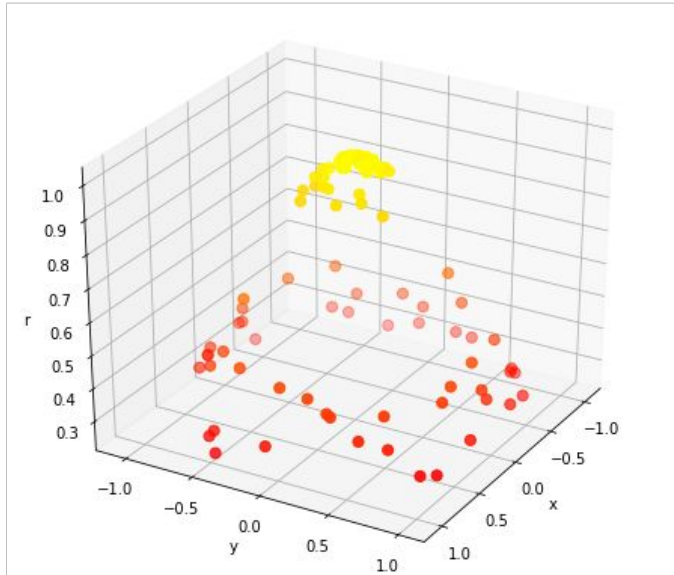
x_1	x_2	x_r
0.25	-0.94	0.38
-0.82	-0.45	0.41
...	...	...



SVM não linear

Podemos ver que com essa transformação os dados originais tornam-se linearmente separáveis.

No Sklearn podemos fazer isso usando a função de kernel RBF



SVM não linear

A função `np.exp(-(Xi**2))` está centralizada na coordenada correta, senão não teríamos visto resultados tão claros e linearmente separáveis. Uma versão não centralizada seria

`np.exp(-(Xi-Xj)**2))`.

- O $-X_j$ causa um deslocamento antes da transformação, isto poderia ser um parâmetro do algoritmo.

Mas, gostaríamos de evitar ter este parâmetro.

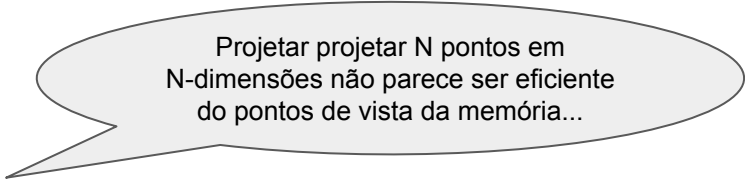
Uma estratégia para esse fim é calcular uma função kernel centrada em **todos** os pontos do conjunto de dados e deixar o algoritmo SVM filtrar os resultados:

$$r_{ij} = \text{np.exp}(-(X_i - X_j)^2)$$

SVM não linear

Se fizermos essa transformação $r_{ij} = \exp(-(X_i - X_j)^2)$ então o algoritmo passa a ter um novo conjunto de dados para classificar:

j	1	2	3	...	N
x_{1j}	x_{11}	x_{12}	x_{13}	...	x_{1N}
x_{2j}	x_{21}	x_{22}	x_{23}	...	x_{2N}
x_{3j}	x_{31}	x_{32}	x_{33}	...	x_{3N}
....	...	...	...	...	..
N	x_{N1}	x_{N2}	x_{N3}	...	x_{NN}



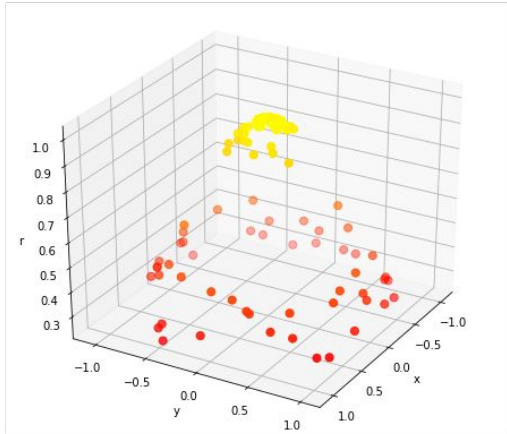
Projetar N pontos em N-dimensões não parece ser eficiente do ponto de vista da memória...

Esse tipo de transformação de função de base é conhecido como transformação de kernel, pois se baseia em uma relação de similaridade (ou kernel) entre cada par de pontos $K(x_i, x_j)$

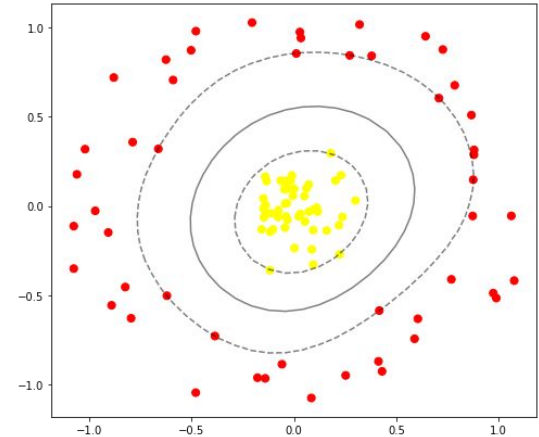
SVM não linear

O Algoritmo do Sklearn faz essa transformação de forma implícita, isso significa que de fato ele não calcula essa matriz toda.

O cálculo implícito chama-se truque do kernel (https://en.wikipedia.org/wiki/Kernel_trick), no qual as operações em baixas dimensões se correspondem às operações em altas dimensões.



$$\hat{y} = \text{sgn} \sum_{i=1}^n w_i y_i k(\mathbf{x}_i, \mathbf{x}')$$



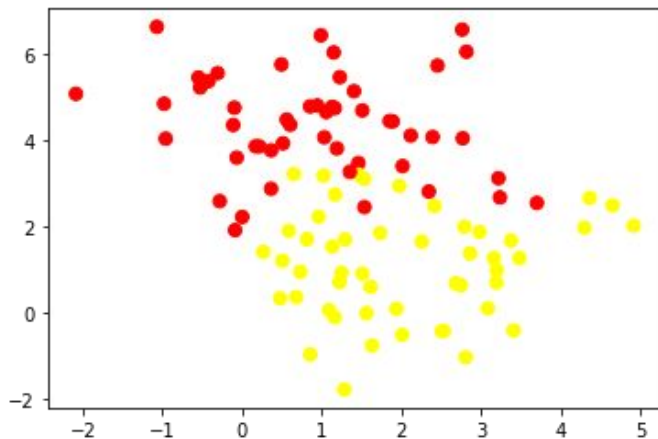
Este truque do kernel está embutido no SVM e é uma das razões pelas quais o método é tão poderoso.

SVM: suavizando as margens

Agora podemos retomar o conceito de regularização (suavização das margens).

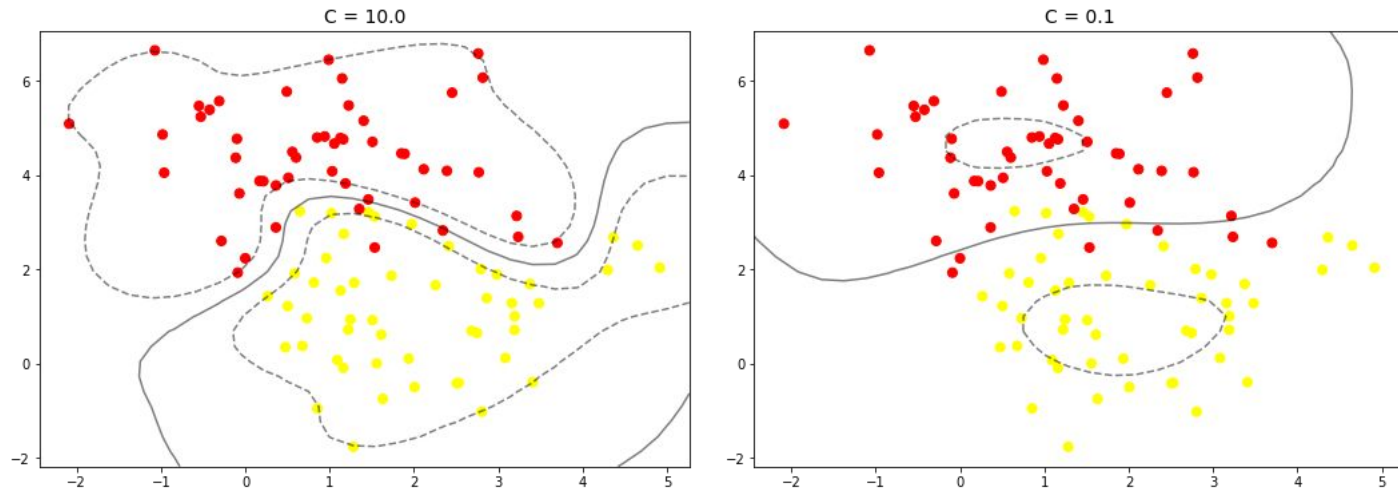
Nossa discussão até agora se concentrou em conjuntos de dados muito limpos, nos quais existe um limite de decisão perfeito.

Mas e se seus dados tiverem alguma sobreposição?



SVM: suavizando as margens

Combinando diferentes kernels com diferentes penalização C podemos gerar coisas maravilhosas!



Experimentar diferentes valores de gamma com kernel RBF.

Sintam o poder
do SVM!

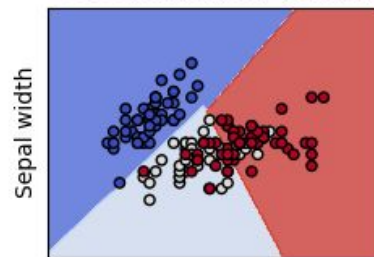


SVM: suavizando as margens

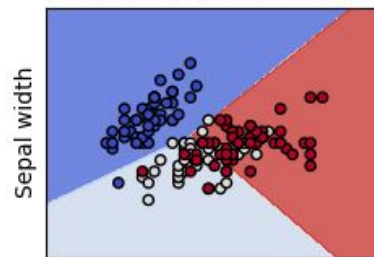
Algumas funções de kernels:

Kernel function	Expression
Liner kernel function	$K(x_i, x_j) = x_i \cdot x_j$
Polynomial kernel function	$K(x_i, x_j) = (x_i \cdot x_j + 1)^d$
Radial basis function (RBF) kernel function	$K(x_i, x_j) = \exp(-\gamma \ x_i - x_j\ ^2)$
Sigmoid kernel function	$K(x_i, x_j) = \tanh(b(x_i, x_j) + c)$

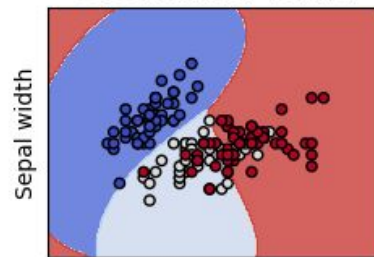
SVC with linear kernel



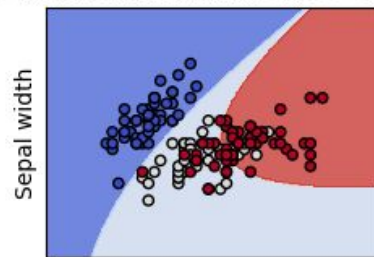
LinearSVC (linear kernel)



SVC with RBF kernel



SVC with polynomial (degree 3) kernel



Atividade prática

Classificador de faces usando SVM

https://colab.research.google.com/drive/12nlu9UsG_8H9tiI5F8OJsVNFrtIPrvwJ?usp=sharing



Colin Powell



George W Bush



George W Bush

Atividade prática

Classificador de faces usando SVM.

- Usaremos o dataset Labeled Faces in the Wild
- Cada imagem contém $[62 \times 47]$ ou quase 3.000 pixels.
- Faremos uma redução no tamanho das imagens (opcional)
- Aplicamos grid search nos parâmetros C e Gamma (com CV=5)
- Tudo dentro de um pipeline

C\Gamma	0.0001	0.0005	0.001	0.005
1	?	?	?	?
5	?	?	?	?
10	?	?	?	?
50	?	?	?	?



Colin Powell



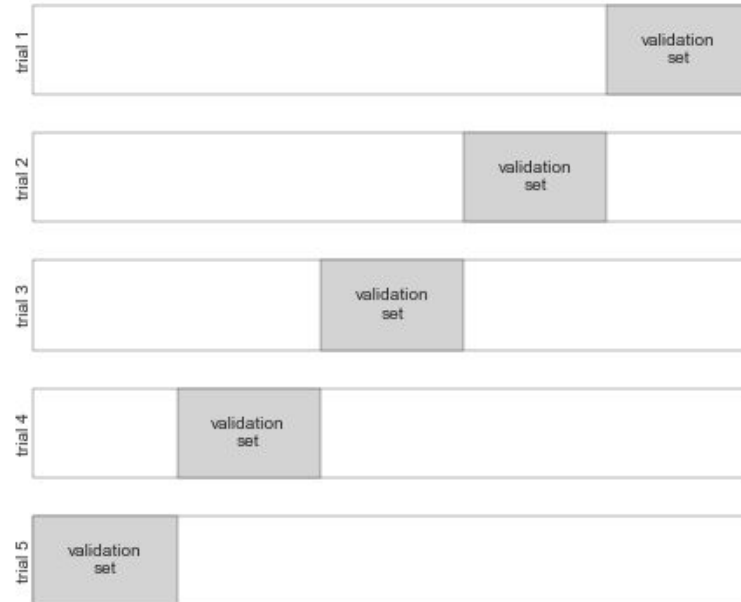
George W Bush



George W Bush

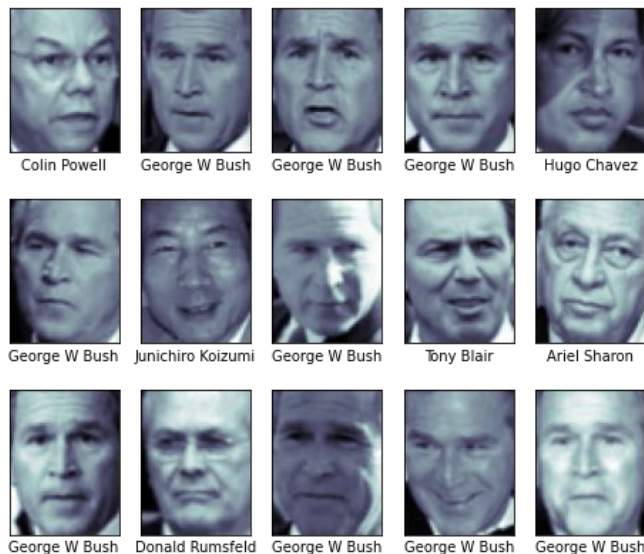
Atividade prática

Cross-validation

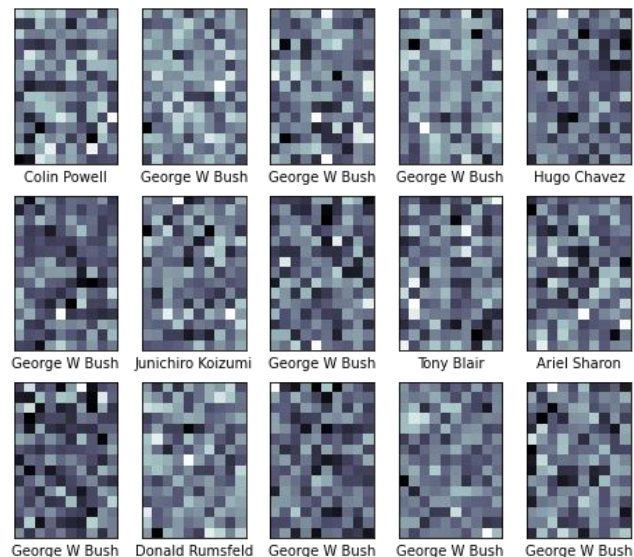


Atividade prática

Redução PCA, as imagens que o classificador vê são:



PCA
→



Resumo de SVM

Vantagens:

- Geralmente encontram poucos vetores de suporte, o que os torna modelos compactos. Uma vez que o modelo é treinado, a fase de previsão é muito rápida.
- A função de decisão depende apenas por pontos próximos à margem, eles funcionam bem com dados de alta dimensão, mesmo dados com mais dimensões do que amostras, o que é desafiador para outros algoritmos.
- Sua integração com diferentes kernels os torna muito versáteis, capazes de se adaptar a muitos tipos de dados.

Resumo de SVM

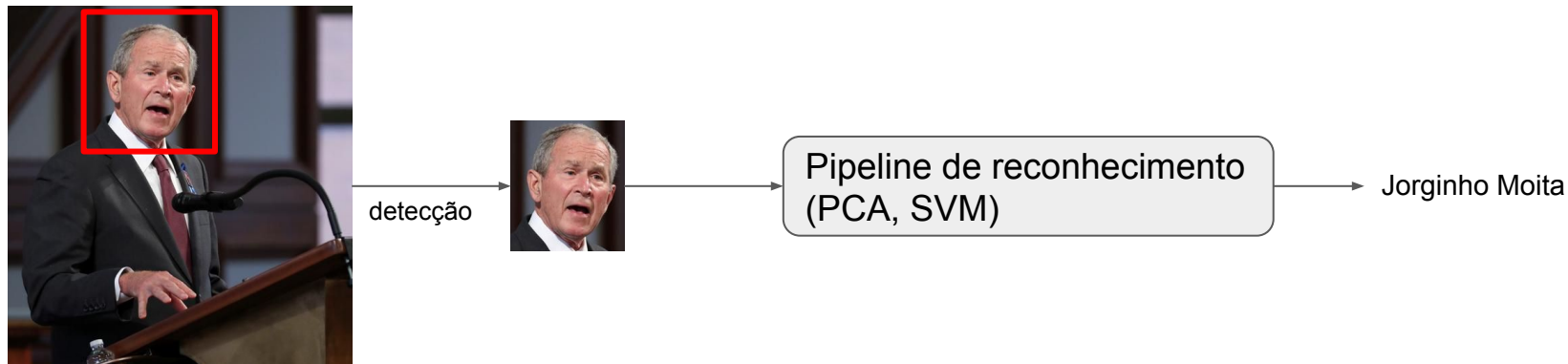
No entanto, os SVMs também têm várias desvantagens:

- O custo computacional escala com o número de amostras $O(N^3)$.
- Os resultados dependem fortemente de uma escolha adequada para o parâmetro C . Escolher C cuidadosamente requer CV, é isso consome mais recursos.
- Os resultados não possuem uma interpretação probabilística direta.

Segunda atividade prática

So far, so good....

Para uma aplicação do mundo real, em que as fotos não vêm pré-cortadas na área do rosto, precisamos usar um algoritmo mais sofisticado para encontrar os rostos primeiro, e classificar depois.



Segunda atividade prática

Precisamos adicionar a detecção dos rostos, então vamos fazer com SVM também!



SVM, mas como?



detecção



Pipeline de reconhecimento
(PCA, SVM)

Jorginho Moita

Scikit-Image

O Scikit-image é uma lib similar ao sklearn para trabalhar com imagens (<https://scikit-image.org>)

Esta lib implementa várias técnicas de extração de atributos a partir de imagens:

- filtros de bordas, cores, texturas, etc.
- HOG
- Rescale, resize and downscale
- RGB to Grayscale
- etc.

HOG

Antes de aplicar o pipeline treinado na detecção de rostos precisamos identificar e recortar o rosto de uma imagem. Existem diferentes formas de como fazer isso.

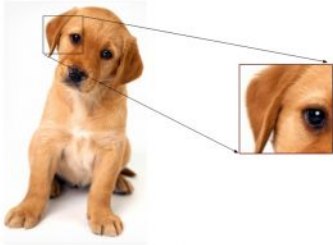
Teremos que aplicar o que chamamos de Engenharia de atributos.

A técnica principal será o Histogram of Oriented Gradients (HOG*), que transforma os pixels da imagem em uma representação vetorial amplamente informativa e robusta às variações de iluminação.

*https://en.wikipedia.org/wiki/Histogram_of_oriented_gradients

HOG

Cálculo dos gradientes x e y:



121	10	78	96	125
48	152	68	125	111
145	78	85	89	65
154	214	56	200	66
214	87	45	102	45

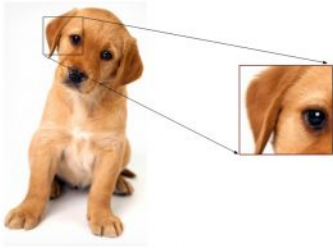
- Change in X direction(G_x) = $89 - 78 = 11$
- Change in Y direction(G_y) = $68 - 56 = 8$

Duas novas imagens são geradas*.

*<https://www.analyticsvidhya.com/blog/2019/09/feature-engineering-images-introduction-hog-feature-descriptor/>

HOG

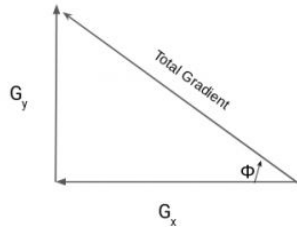
Cálculo dos gradientes x e y:



121	10	78	96	125
48	152	68	125	111
145	78	85	89	65
154	214	56	200	66
214	87	45	102	45

- Change in X direction(G_x) = $89 - 78 = 11$
- Change in Y direction(G_y) = $68 - 56 = 8$

Cálculo da orientação e magnitude:

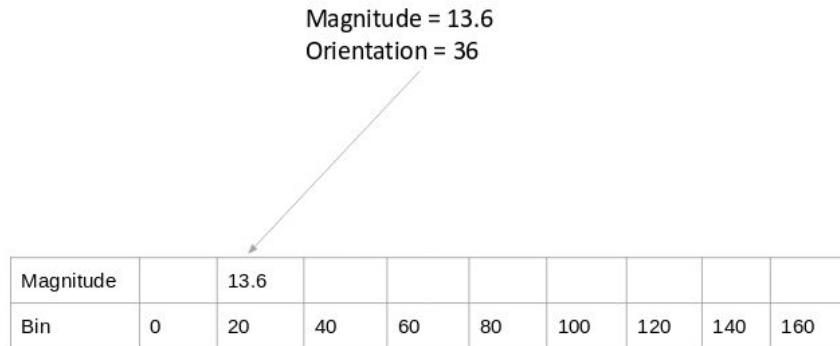


- Total Gradient Magnitude = $\sqrt{[(G_x)^2 + (G_y)^2]}$
- $\Phi = \text{atan}(G_y / G_x)$

Duas novas imagens são geradas (M e Φ)

HOG

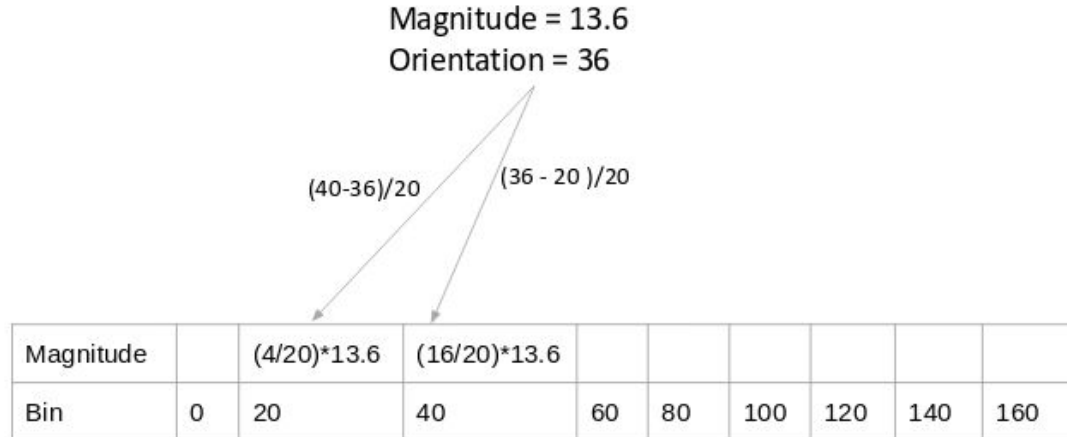
Os Φ podem ser divididos em um histograma com 9 bins e em cada bin pode ser colocada a magnitude do gradiente:



HOG

Os Φ podem ser divididos em um histograma com 9 bins e em cada bin pode ser colocada a magnitude do gradiente.

De fato, a magnitude do gradiente é ponderada pela proximidade com o intervalo do bin:



HOG

Esse procedimento é repetido para cada célula da imagem:



Cada célula pode ter um tamanho de 8x8 pixels, e seria gerado um vetor de 9x1 para cada uma delas.

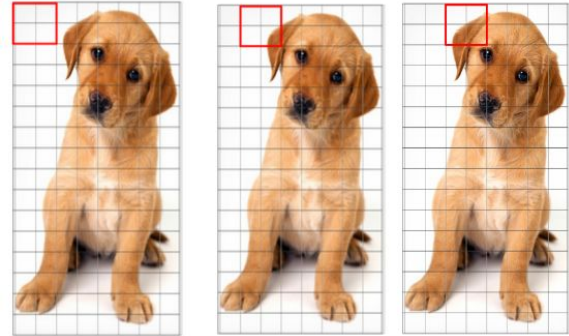
HOG

A normalização é realizada escolhendo blocos maiores, por exemplo 16x16 pixel, que corresponde a um vetor de features com tamanho $4 \times 9 \times 1 = 36 \times 1$

Para normalizar essa matriz, vamos dividir cada um desses valores pela raiz quadrada da soma dos quadrados dos valores.

$$k = \sqrt{(a_1)^2 + (a_2)^2 + (a_3)^2 + \dots + (a_{36})^2}$$

$$\text{Normalised Vector} = \left(\frac{a_1}{k}, \frac{a_2}{k}, \frac{a_3}{k}, \dots, \frac{a_{36}}{k} \right)$$



A normalização reduz a variação nas features causada pelas diferenças de iluminação.

HOG

Ao finalizar este procedimento vamos ter 105 (7x15) blocos de 16x16 cada um, os quais geraram 36x1 features, portanto o vetor final de features será $105 \times 36 \times 1 = 3780$.

No método `skimage.features.hog()` :

- `orientations=8` é a quantidade de bins do histograma
- `pixels_per_cell=(8, 8)` o tamanho de cada célula
- `cells_per_block=(2, 2)` o agrupamento de células para normalização



HOG

Resultado

Input image



Histogram of Oriented Gradients



Face detection

Uma vez que entendemos o descritor da imagem vamos criar o detector de rostos.

As etapas são as seguintes:

1. Obter um conjunto de imagens de rostos para constituir amostras de treinamento “positivas”.
2. Obter um conjunto de imagens de “não” faces para constituir amostras de treinamento “negativas”.
3. Extraia o HOG desses exemplos de treinamento.
4. Treinar um classificador SVM nessas amostras.

Detecção para uma imagem “desconhecida”:

1. Passar uma janela deslizante pela imagem aplicando o SVM para avaliar se essa janela contém um rosto.
2. Se as detecções se sobrepõem, combine-as em uma única janela.

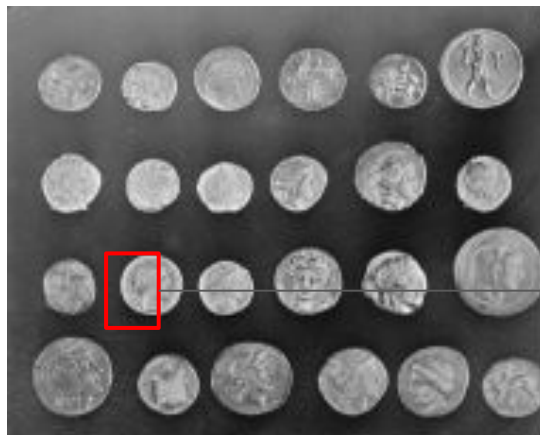
Face detection

Para compor a classe negativa:

1. Escolher uma imagem qualquer
2. reascale
3. faz um crop relatório de 62x47
4. repeat

Dataset negativo:

1. Extrair HOG

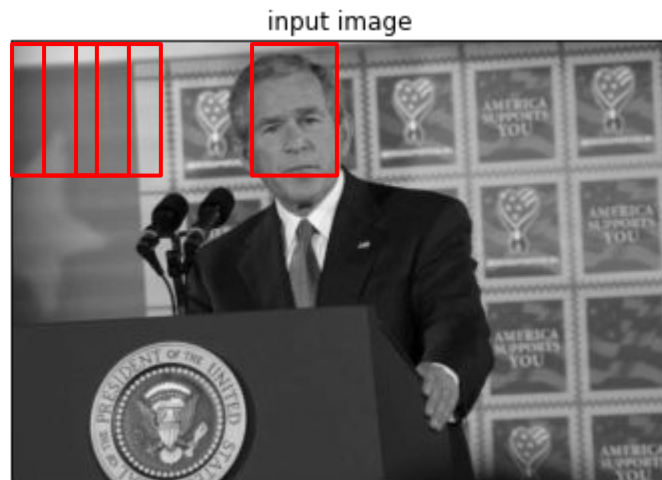


amostra da
classe negativa

Face detection

Hora de testar o detector.

1. Fazemos um “sliding window” de tamanho 64x47
2. Extraímos HOG de cada window
3. Passamos para o SVM binário

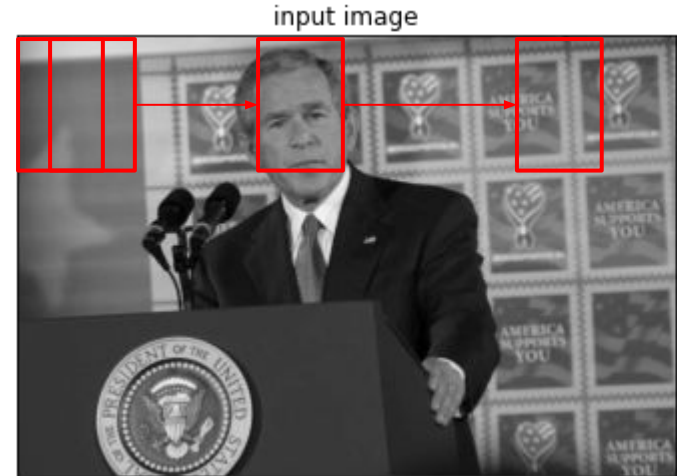


Face detection

Hora de testar o detector.

1. Fazemos um “sliding window” de tamanho 64x47
2. Extraímos HOG de cada window
3. Passamos para o SVM binário

Resultado:

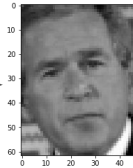


Face detection and Recognition

Finalmente:



detecção



Pipeline de reconhecimento
(PCA, SVM)

Jorginho Moita

Face detection

Existem várias melhorias que poderiam ser feitas:

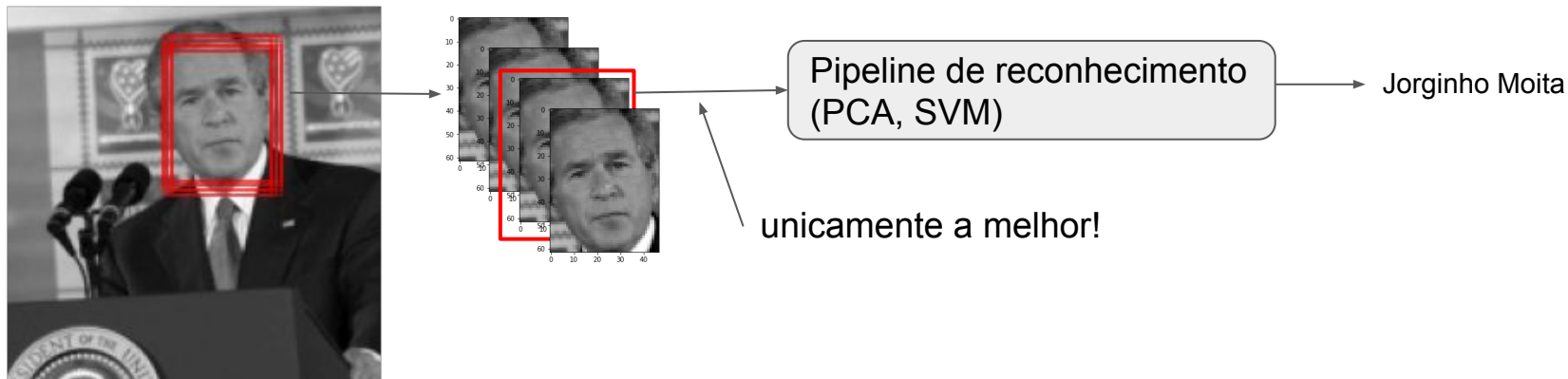
1. Nosso conjunto de treinamento, especialmente para características negativas, não é muito completo
2. Nosso pipeline atual detecta apenas em uma escala
3. Devemos combinar patches de detecção sobrepostos

Avanços mais recentes em Deep Learning:

1. HOG e outros métodos de extração de recursos procedimentais para imagens não são mais técnicas de última geração.
2. Em vez disso, muitos pipelines modernos de detecção de objetos usam variantes de redes neurais profundas: uma maneira de pensar em redes neurais é que elas são um estimador que determina estratégias de extração de atributos ideais dos dados, em vez de depender da intuição do usuário.

Atividade prática

Repetir a atividade da aula, mas escolhendo a melhor janela do detector e passar unicamente essa janela para o modelo de reconhecimento:



Usar como critério aquela janela que maximiza a distância à função de decisão do modelo de detecção (o score).

Enviar pelo Colab.